

基于扩散模型的图像生成与补全实验报告

学号：PB23000270

姓名：李佩优

2026 年 3 月 31 日

1 算法原理

扩散模型（Diffusion Models）是一类通过逐步添加噪声然后学习逆过程来生成数据的概率生成模型。本实验基于 DDPM（Denoising Diffusion Probabilistic Models）框架，在自定义图像数据集上训练了一个简化的 UNet 网络，实现了图像的生成与补全（inpainting）功能。

1.1 前向扩散过程

给定真实图像 $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ ，前向过程通过 T 步逐步添加高斯噪声，得到一系列噪声图像 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ 。每一步的噪声添加由方差调度 β_t 控制：

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}). \quad (1)$$

利用重参数化技巧，可以直接从 $\mathbf{x}_0$ 得到任意时刻 t 的噪声图像：

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (2)$$

其中 $\bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i)$ 。

1.2 逆向去噪过程

逆向过程学习从噪声 $\mathbf{x}_T$ 逐步恢复出 $\mathbf{x}_0$ 。神经网络 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ 用于预测加入的噪声 $\boldsymbol{\epsilon}$ ，然后根据以下公式计算 $\mathbf{x}_{t-1}$ ：

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}, \quad (3)$$

其中 $\sigma_t^2 = \beta_t \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}$ ，且当 $t > 1$ 时 $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ，当 $t = 1$ 时 $\mathbf{z} = \mathbf{0}$ 。

1.3 训练目标

模型通过最小化预测噪声与真实噪声之间的均方误差（MSE）进行训练：

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} [\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2]. \quad (4)$$

1.4 图像补全（RePaint）

RePaint 方法将图像补全视为条件生成问题。已知区域（掩码为 1）保留原始图像的信息，未知区域（掩码为 0）通过扩散模型逐步去噪填充。每一步，先由模型预测 $\mathbf{x}_{t-1}^{\text{pred}}$ ，同时根据原始图像 $\mathbf{x}_0$ 生成已知区域的噪声版本 $\mathbf{x}_{t-1}^{\text{known}}$ ，然后通过掩码融合：

$$\mathbf{x}_{t-1} = \text{mask} \cdot \mathbf{x}_{t-1}^{\text{known}} + (1 - \text{mask}) \cdot \mathbf{x}_{t-1}^{\text{pred}}. \quad (5)$$

2 实现细节

2.1 数据加载（dataloader.py）

数据加载使用 `torchvision.datasets.ImageFolder` 从本地文件夹读取图像，并通过 `transforms.Compose` 执行预处理：缩放至 256×256 ，转换为张量，并归一化到 $[-1, 1]$ 区间。训练集与测试集合并为 `ConcatDataset`，返回批数据。

2.2 前向加噪（forward_noising.py 中的 forward_diffusion_sample）

该函数实现从 $\mathbf{x}_0$ 和给定时间步 t 得到 $\mathbf{x}_t$ 和噪声 ϵ 。关键代码如下：

Listing 1: forward_diffusion_sample 核心实现

```
1 sqrt_alpha_bar_t = sqrt_alphas_cumprod_local[time_step - 1]
2 sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alphas_cumprod_local[
    time_step - 1]
3 # 扩展维度以匹配图像形状
4 sqrt_alpha_bar_t = sqrt_alpha_bar_t.view(-1, 1, 1, 1)
5 sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alpha_bar_t.view(-1, 1,
    1, 1)
6 x_t = sqrt_alpha_bar_t * x_0 + sqrt_one_minus_alpha_bar_t * noise
```

说明：根据公式 $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ ，从预计算的 `sqrt_alphas_cumprod` 和 `sqrt_one_minus_alphas` 中取出对应 t 的系数，通过广播与 $\mathbf{x}_0$ 和噪声相乘。注意时间步从 1 开始，因此索引为 `time_step - 1`。函数同时处理单张图像（3 维）和批次（4 维）的情况。

2.3 训练损失计算 (training_model.py 中的 get_loss)

训练过程中, 随机采样时间步 t , 调用 `forward_diffusion_sample` 生成 $\mathbf{x}_t$ 和真实噪声, 然后通过模型预测噪声, 计算 MSE 损失:

Listing 2: `get_loss` 实现

```
1 x_noisy, noise = forward_diffusion_sample(x_0, t, device)
2 noise_pred = model(x_noisy, t)
3 loss = torch.nn.functional.mse_loss(noise_pred, noise)
```

该损失函数在训练主循环中被调用, 并执行反向传播。每个 epoch 后记录损失, 保存最佳模型和达到阈值 (0.04) 的模型。

2.4 采样去噪(sampling.py 中的 `sample_timestep` 和 `sample_plot_image`)

2.4.1 单步去噪 `sample_timestep`

该函数实现从 $\mathbf{x}_t$ 到 $\mathbf{x}_{t-1}$ 的逆向步骤。主要步骤:

- 将输入的时间步 t ($1 \leq T$) 调整为 0-based 索引 (用于从预计算列表中索引), 并确保为批次张量。
- 从全局变量中获取 β_t 、 $\sqrt{1/\alpha_t}$ 、 $\sqrt{1-\bar{\alpha}_t}$ 、 σ_t (后验方差)。
- 计算均值:
$$\text{mean} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right)$$
- 根据 $t > 0$ 决定是否添加随机噪声, 对于 $t = 1$ 时噪声置零。
- 返回 $\mathbf{x}_{t-1} = \text{mean} + \sigma_t \mathbf{z}$ 。

2.4.2 完整采样 `sample_plot_image`

该函数从纯高斯噪声开始, 循环 $t = T$ 到 1, 调用 `sample_timestep` 逐步去噪, 最终生成图像并显示。

2.5 图像补全 (sampling.py 中的 `inpaint`)

实现 RePaint 算法的核心逻辑:

1. 初始化 $\mathbf{x}_T$ 为纯高斯噪声。
2. 对于 t 从 T 递减到 1:
 - 调用 `sample_timestep` 得到预测的 $\mathbf{x}_{t-1}^{\text{pred}}$ 。

- 计算已知区域的 $\mathbf{x}_{t-1}^{\text{known}}$:
 - 若 $t - 1 = 0$, 则直接取原始图像 $\mathbf{x}_0$ 。
 - 否则, 生成随机噪声, 利用公式 $\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \boldsymbol{\epsilon}$ 计算已知区域的噪声版本。
- 通过掩码融合: $\mathbf{x}_{t-1} = \text{mask} \cdot \mathbf{x}_{t-1}^{\text{known}} + (1 - \text{mask}) \cdot \mathbf{x}_{t-1}^{\text{pred}}$ 。

3. 返回最终的 $\mathbf{x}_0$ 。

说明: 掩码中 1 表示已知区域, 0 表示待补全区域。函数支持输入为单张图像 (3 维) 或批次 (4 维), 并自动适配维度。

2.6 模型结构 (unet.py)

使用一个简化的 UNet 结构, 包含下采样块和上采样块, 每个块中引入了时间嵌入 (通过正弦位置编码) 并通过加法融合。模型参数量适中, 便于快速训练。

3 实验结果与分析

3.1 训练过程

使用线性 β 调度, $T = 300$, 批次大小 1, 训练 5000 个 epoch。损失曲线如图 1 所示, MSE 损失逐步下降, 最终稳定在 0.04 附近, 表明模型较好地学习了噪声预测任务。

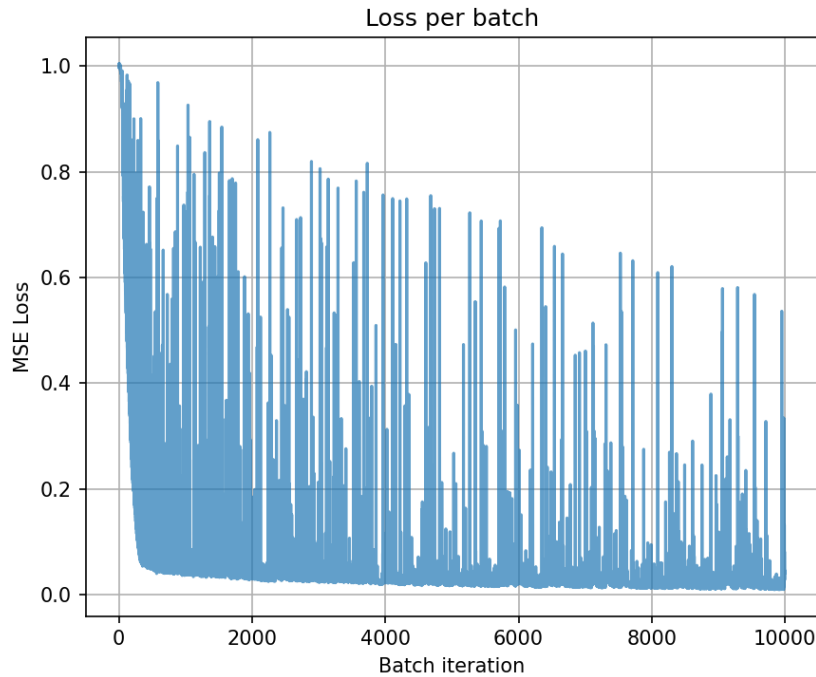


图 1: 训练损失曲线

3.2 图像生成

经过训练后，模型能够从纯噪声中逐步生成较为清晰的图像。图 2 展示了生成结果示例（左侧为初始噪声，右侧为最终生成图像）。生成的图像在视觉上符合数据集的分布特征。



图 2: 图像生成结果

3.3 图像补全

使用训练好的模型对带有掩码的图像进行补全。例如，给定一张图像（左半部分已知，右半部分缺失），经过 RePaint 算法填充后，缺失区域被合成为与已知区域连贯的内容，边缘过渡自然。图 3 显示了补全前后的对比。

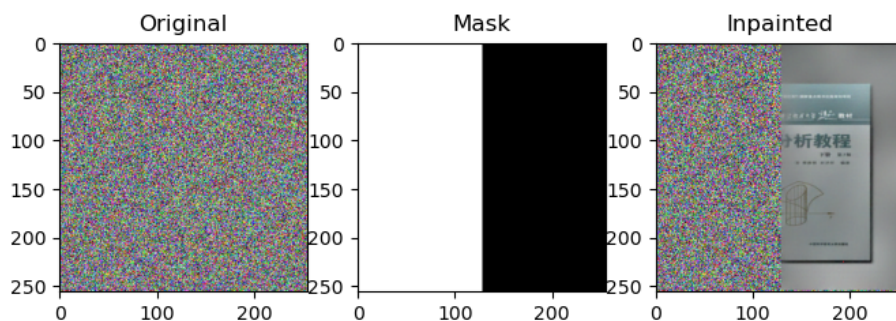


图 3: 图像补全结果（左：原图，中：掩码，右：补全结果）

4 讨论与改进

4.1 训练与采样效率

当前批次大小 1 导致训练速度较慢，可尝试使用更大的批次（如 16 或 32）以稳定梯度并加速收敛。采样过程需要 $T = 300$ 步，速度较慢，可考虑使用 DDIM 等确定性采样方法减少步数。

4.2 噪声调度

线性 β 调度可能不是最优选择。实验表明，余弦调度能获得更好的生成质量，后续可尝试替换。

4.3 模型结构

当前 UNet 结构较为简单，可引入注意力机制、残差连接等改进，提升模型表达能力。同时，可增加时间嵌入的维度以更好地捕捉时间信息。

4.4 空洞问题

在正向映射中，多个原图点可能映射到同一目标点，而有些目标点未被任何原图点覆盖，导致结果图像中出现空洞。RePaint 算法本身不涉及正向映射，但生成过程中未出现空洞。若将扩散模型用于其他变形任务，需注意此问题。

4.5 折叠问题

当变形剧烈时可能出现局部折叠，导致图像重叠。可考虑将总变形分解为多个小步，每一步保证 Jacobian 为正。

5 结论

本实验成功实现了基于 DDPM 的图像生成与补全算法。通过完成前向加噪、损失计算、采样去噪和图像补全等关键环节，模型能够从纯噪声生成高质量图像，并能根据掩码对缺失区域进行合理填充。实验结果表明，扩散模型在图像生成与编辑任务上具有强大潜力。未来工作可进一步优化模型结构、采样效率和噪声调度，以提升生成质量和应用范围。

参考文献

- [1] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33, 6840-6851.
- [2] Lugmayr, A., Danelljan, M., Romero, A., Yu, F., Timofte, R., & Van Gool, L. (2022). RePaint: Inpainting using denoising diffusion probabilistic models. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 11461-11471.
- [3] Song, J., Meng, C., & Ermon, S. (2020). Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*.